AM5920: Deep learning for Mechanics

Assignment 1

Mohamed Mafaz
AM25M009

Problem 1

This implementation was done from complete scratch, including dataloaders, as **mini batch** approach was used as its the most common when it comes to machine learning. with **BATCH SIZE** = 15.

regularization was used: [0, 0.001, 0.005, 0.01, 0.1, 0.5],

Only numpy and matplotlib was used for this problem.

The function choosen for this problem:

$$z = \sin x + \cos y + \text{noise} \tag{1}$$

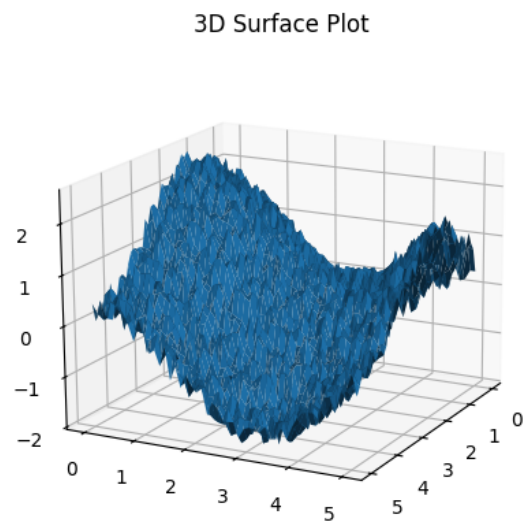

Figure 1: Function chosen

noise was added in the domain [-1, 1]
the general formula for linear regression with regularziation:

$$\mathcal{L}(w, b) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - (w^T x_i + b) \right)^2 + \lambda \|w\|_2^2 \tag{2}$$

Convergence criteria

for training:

$$|\text{epoch_loss_train} - \text{old_train_loss}| < 0.4 \quad \text{and} \quad \text{epoch_loss_train} < 0.4$$

Problem 1 continued on next page. . .

for testing:

$$|\text{epoch_loss_test} - \text{old_test_loss}| < 0.4 \quad \text{and} \quad \text{epoch_loss_test} < 0.4$$

Convergence for each lambdas

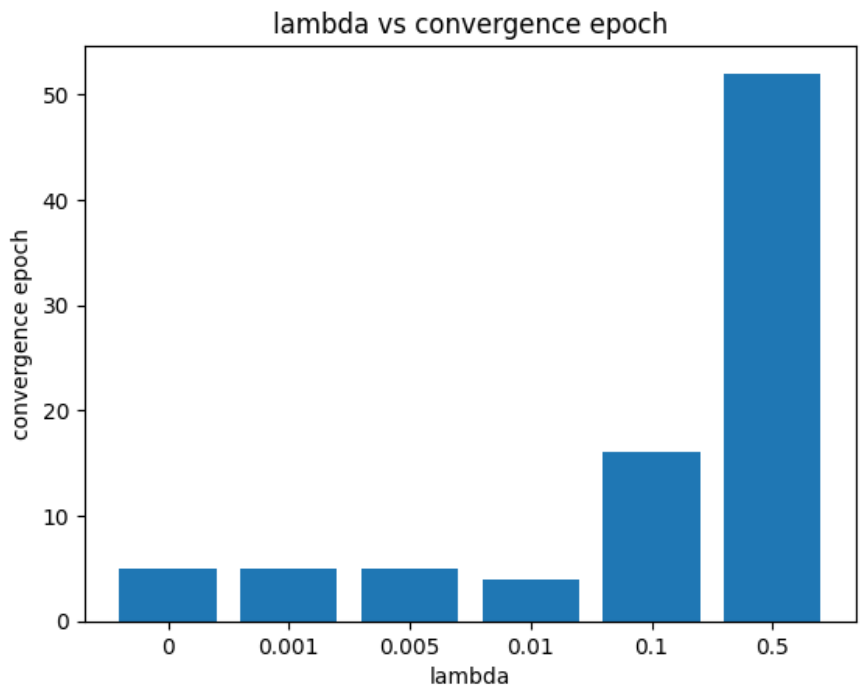

Figure 2: Lambdas vs Convergence for Training Data

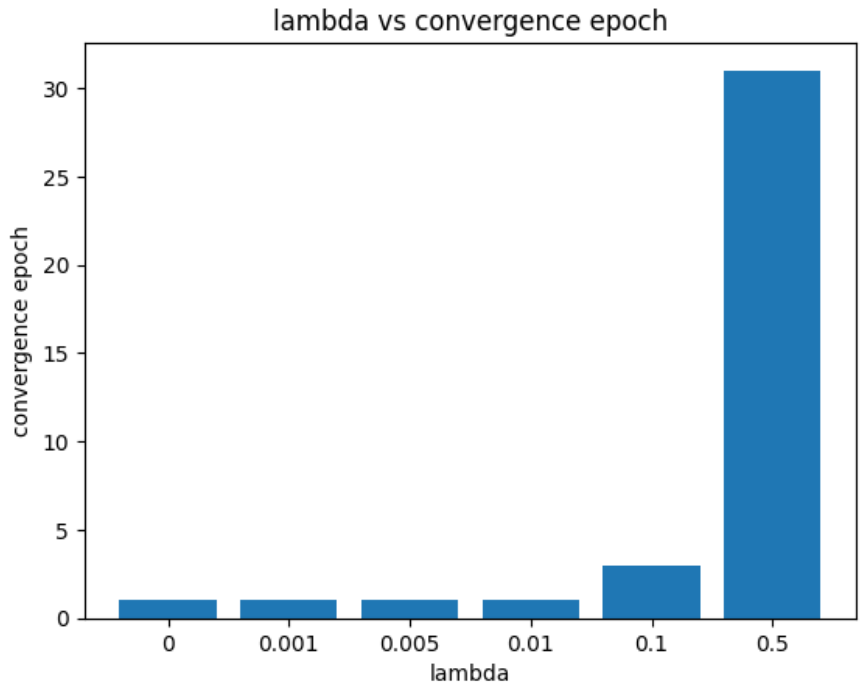

Figure 3: Lambdas vs Convergence for Testing Data

trend observed: As λ increases $\rightarrow$ training loss increases
Because:

- Large λ forces weights toward zero
- Model becomes less flexible
- It cannot perfectly fit training data
- So error increases

Tracking Epoch loss

multiple regularization were applied and studied:

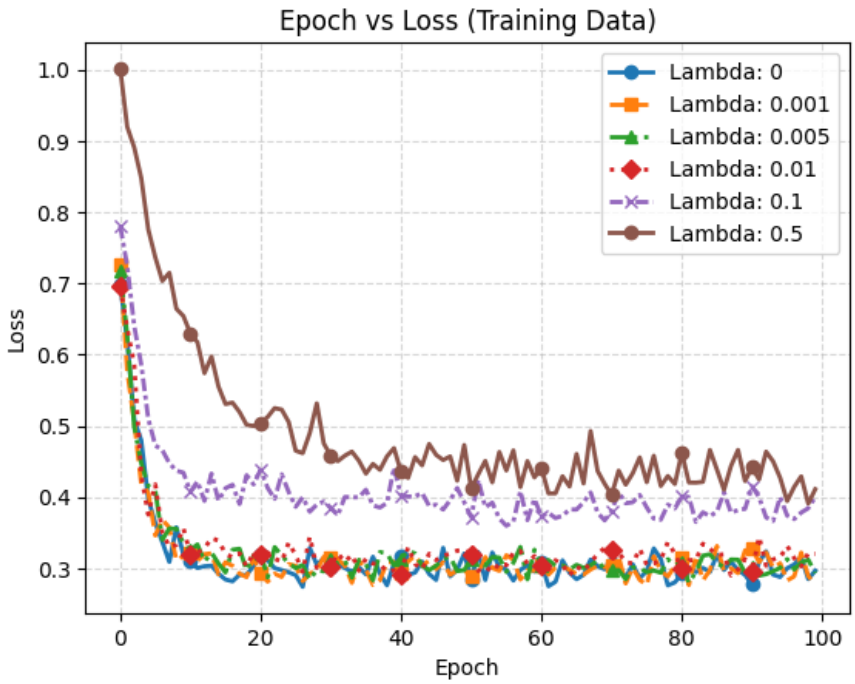

Figure 4: lamdas vs Loss Training data

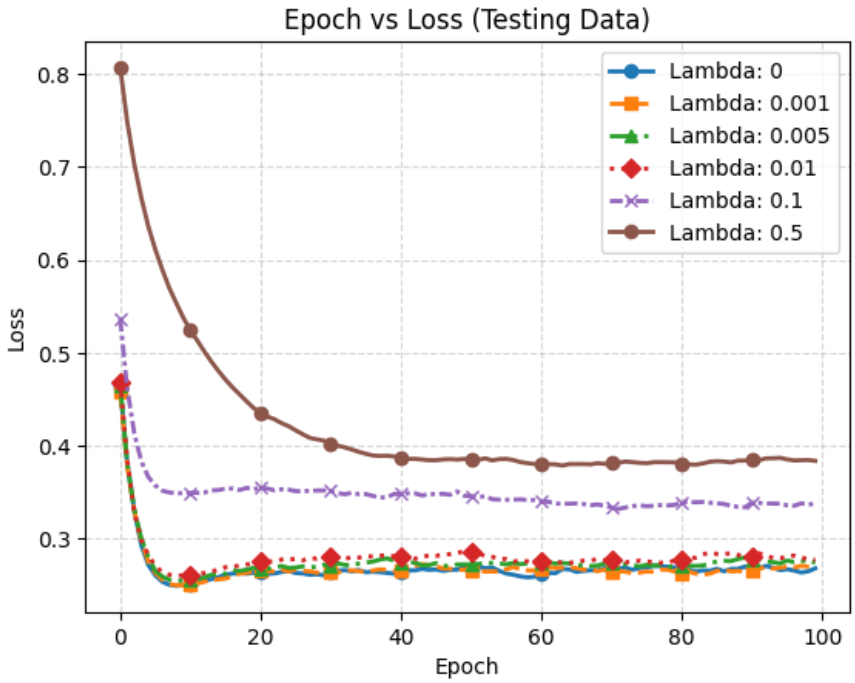

Figure 5: lamdas vs Loss Testing data

From the training and testing loss curves, it is observed that increasing the regularization parameter λ leads to higher training loss, as stronger regularization constrains the weights and reduces model

flexibility. For very small values of λ (0, 0.001, 0.005, 0.01), both training and testing losses converge quickly to similar low values, indicating good generalization. However, for larger values such as $\lambda = 0.1$ and especially $\lambda = 0.5$, the model underfits, resulting in significantly higher training and testing losses. This behaviour reflects the bias–variance tradeoff, where moderate regularization improves stability while excessive regularization degrades performance.

Parameters vs Loss, 3D view

since there requires 2 input and 1 output, weight matrix is 1x2, w_1 and w_2 and b_1 are the total paramter, for the sake of 3d view, I am ignoring bais term

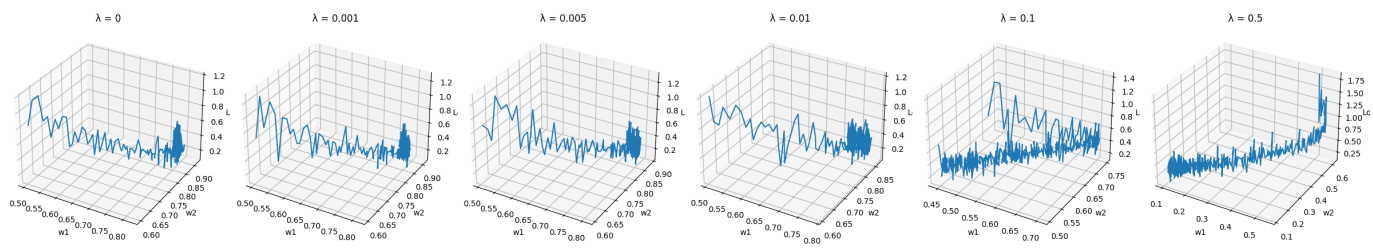

Figure 6: Parameters vs Loss

L2 loss surface

the entire dataset was taken as one batch and performed matrix multiplication over multiple weight values to form a mesg/surface

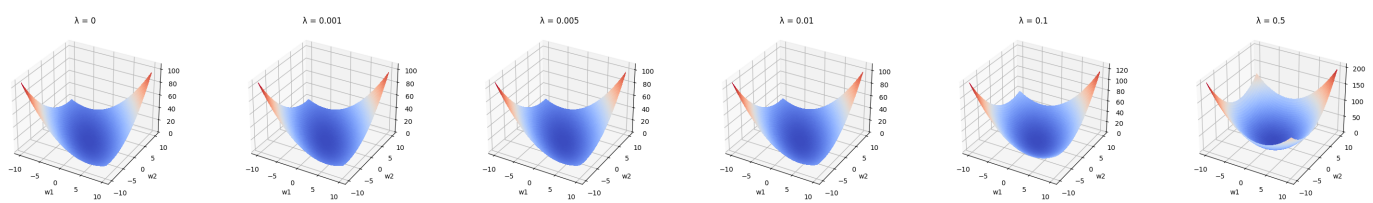

Figure 7: L2 Loss surface

Conclusion and Justification

In this study, linear regression with L2 regularization was implemented from scratch using mini-batch training to approximate a nonlinear noisy function. The experiments clearly demonstrate the effect of the regularization parameter (λ) on model performance.
For small values of λ (0 to 0.01), the model achieves low training and testing loss, indicating good generalization and stable convergence. As λ increases (0.1 and 0.5), both training and testing losses increase significantly, showing underfitting due to excessive constraint on the weights.

This behavior aligns with the bias–variance tradeoff: moderate regularization improves stability and prevents overfitting, while overly large regularization reduces model flexibility and increases bias. The loss surface visualization further confirms that L2 regularization smooths the optimization landscape, guiding the weights toward smaller magnitudes and promoting more stable convergence. Overall, the results justify that a carefully chosen regularization parameter is essential for balancing model flexibility and generalization performance.

Problem 2

This implementation was done using Pytorch, as **mini batch** approach was used as its the most common when it comes to machine learning. with **BATCH SIZE** = 10.
The model architecture is defined as:
Let d be the input dimension, W the width of each hidden layer, and L the number of hidden layers.

$$\begin{aligned} \textbf{Input Layer:} \quad & \mathbb{R}^d \to \mathbb{R}^W \\ \textbf{Hidden Layers:} \quad & L \text{ fully connected layers} \\ & \mathbb{R}^W \to \mathbb{R}^W \\ \textbf{Output Layer:} \quad & \mathbb{R}^W \to \mathbb{R}^1 \end{aligned}$$

Here d = 2.
More formally,

$$f(x) = W_{out}\,\sigma\Big(W_{10}\,\sigma(\ldots\sigma(W_1 x + b_1)\ldots) + b_{10}\Big) + b_{out}$$

where $\sigma(\cdot)$ denotes the activation function. In this work, we consider both sigmoid and ReLU activations.
For sigmoid activation,

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{3}$$

For ReLU activation,

$$\sigma(z) = \max(0, z). \tag{4}$$

Concerned Parameter

I will be messing with Width, Depth, Learning Rate. once optimal W, D, LR are justified, I will be messing with Batch size and compare convergence

Convergence criteria

for training:

$$|\text{epoch_loss_train} - \text{old_train_loss}| < 0.4 \quad \text{and} \quad \text{epoch_loss_train} < 0.15$$

for testing:

$$|\text{epoch_loss_test} - \text{old_test_loss}| < 0.4 \quad \text{and} \quad \text{epoch_loss_test} < 0.15$$

Problem 2 continued on next page. . .

Grid Search

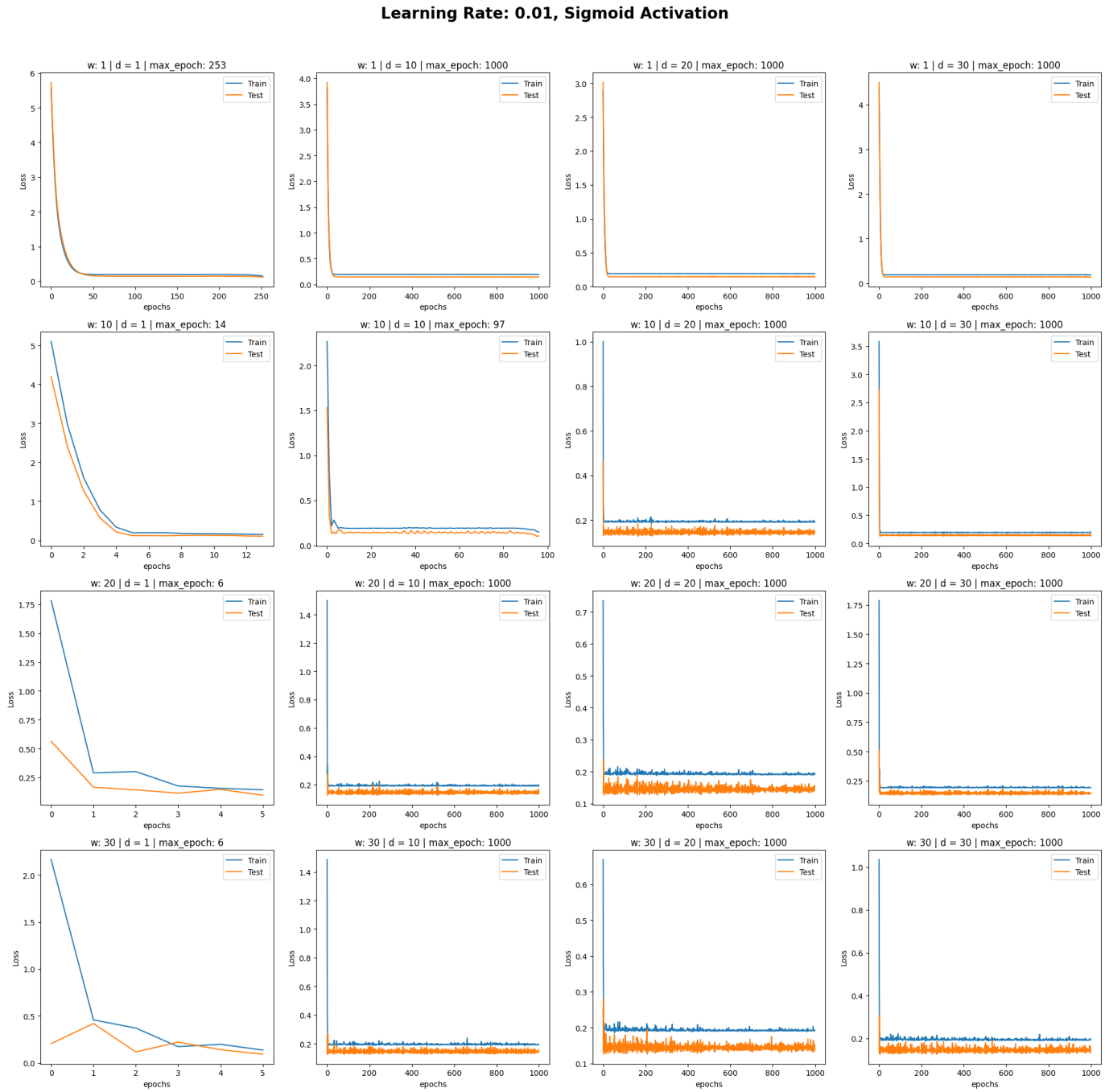

Figure 8: Grid Search for lr=0.01, Sigmoid Activation

Problem 2 continued on next page. . .

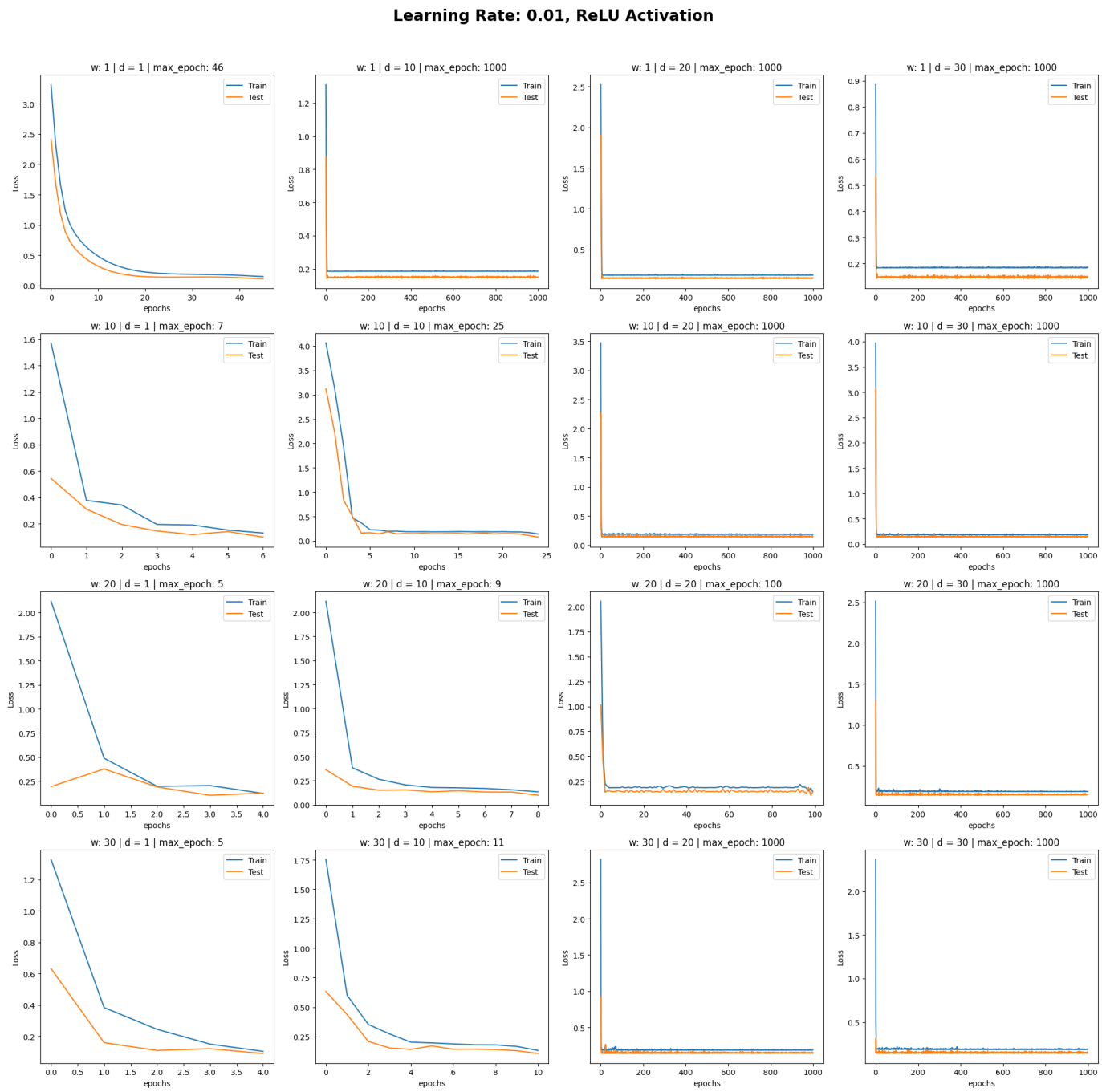

Figure 9: Grid Search for lr=0.01, ReLU Activation

Justification for Learning Rate $\eta = 0.01$

For shallow networks, the loss decreases rapidly and converges within a small number of epochs, indicating that the learning rate $\eta = 0.01$ is appropriate for effective optimization.
However, as network depth increases, the convergence behaviour depends strongly on the activation function used. With sigmoid activation, deeper models converge significantly more slowly or may

fail to reach the desired loss threshold within the maximum number of epochs. This is because the derivative of the sigmoid is bounded above by 0.25, and during backpropagation the gradients are multiplied across layers. For depth L, the gradient magnitude scales approximately as $(0.25)^L$, leading to vanishing gradients and extremely small parameter updates.

In contrast, when ReLU activation is used, deeper networks converge much faster and more consistently. Since the derivative of ReLU is 1 in its active region, gradient magnitudes are better preserved across layers, allowing efficient training even for larger depths.

Therefore, while $\eta = 0.01$ is generally suitable, the observed differences in convergence across depths are primarily influenced by the activation function rather than the learning rate itself.

Problem 2 continued on next page. . .

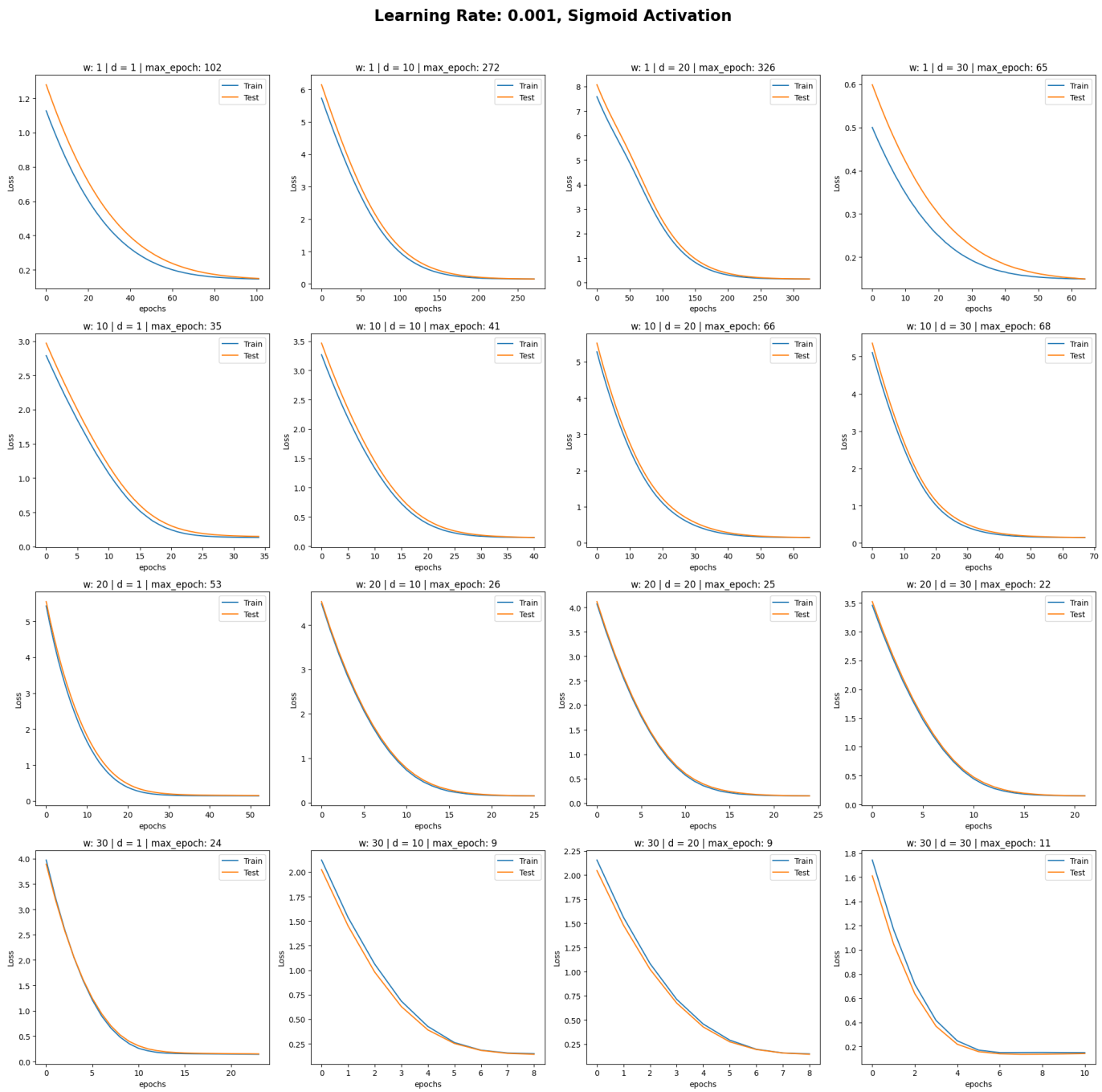

Figure 10: Grid Search for lr=0.001, Sigmoid Activation

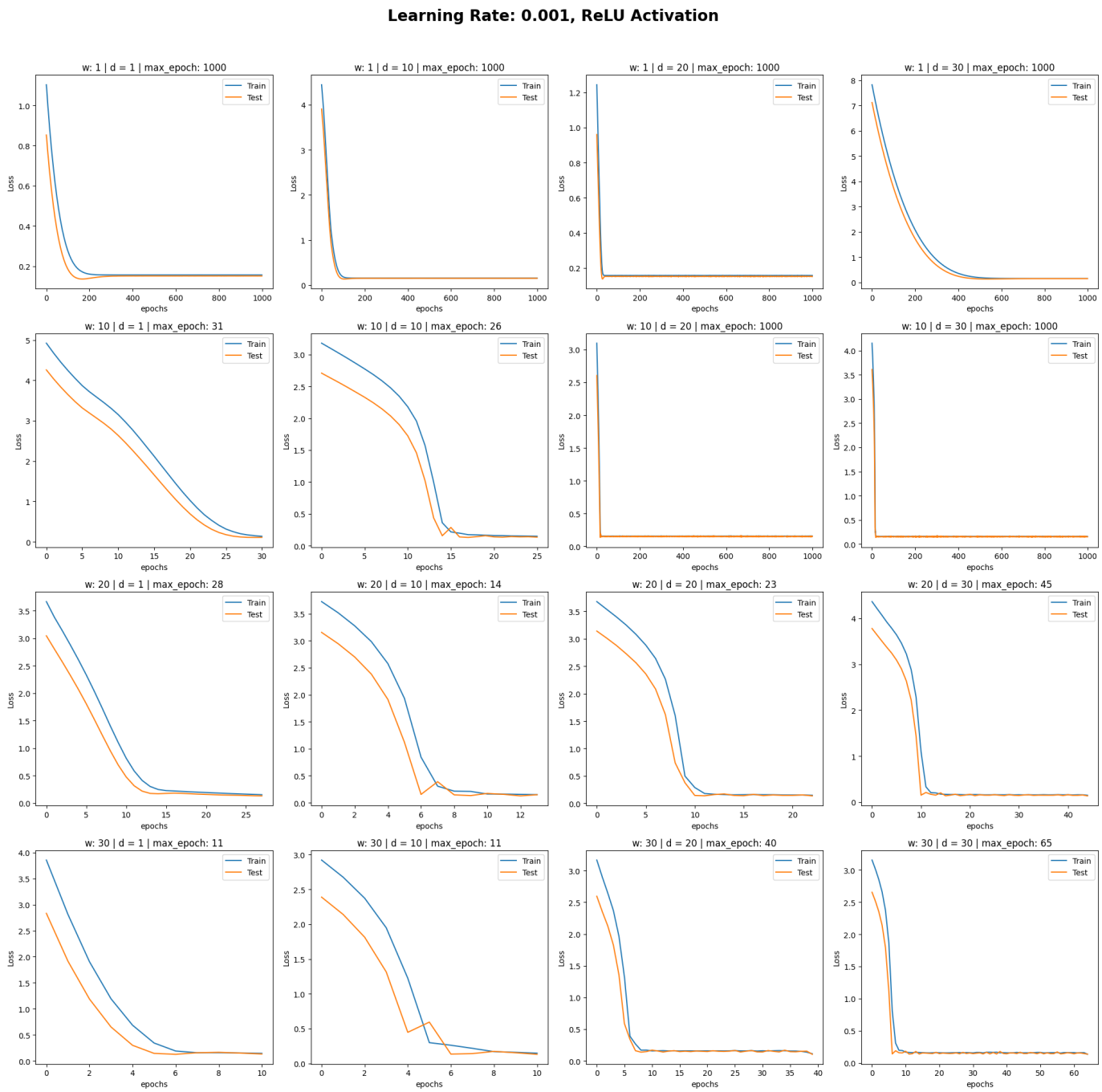

Figure 11: Grid Search for lr=0.001, ReLU Activation

Justification for Learning Rate $\eta = 0.001$

When using $\eta = 0.001$, the convergence behaviour becomes more sensitive to both network depth and the choice of activation function.
With sigmoid activation, most architectures exhibit smooth and stable loss decay across depths. Although convergence may require more epochs for deeper networks, training remains stable without

oscillations. The smaller learning rate prevents large parameter updates, which is beneficial in deep sigmoid networks where gradients are already small due to repeated multiplication across layers. Hence, $\eta = 0.001$ provides stable optimization, albeit sometimes at the cost of slower convergence. In contrast, with ReLU activation, several deeper networks fail to converge within the maximum number of epochs or converge very slowly. Since ReLU preserves gradient magnitude in its active region, the effective updates become directly proportional to the learning rate. A smaller learning rate therefore leads to very small parameter updates, causing slow optimization and delayed convergence. Thus, while $\eta = 0.001$ ensures stable training for sigmoid-based architectures, it can be overly conservative for ReLU networks, resulting in unnecessarily slow convergence. The results indicate that this learning rate is stable but not always optimal, particularly for ReLU-based deep models.

Problem 2 continued on next page. . .

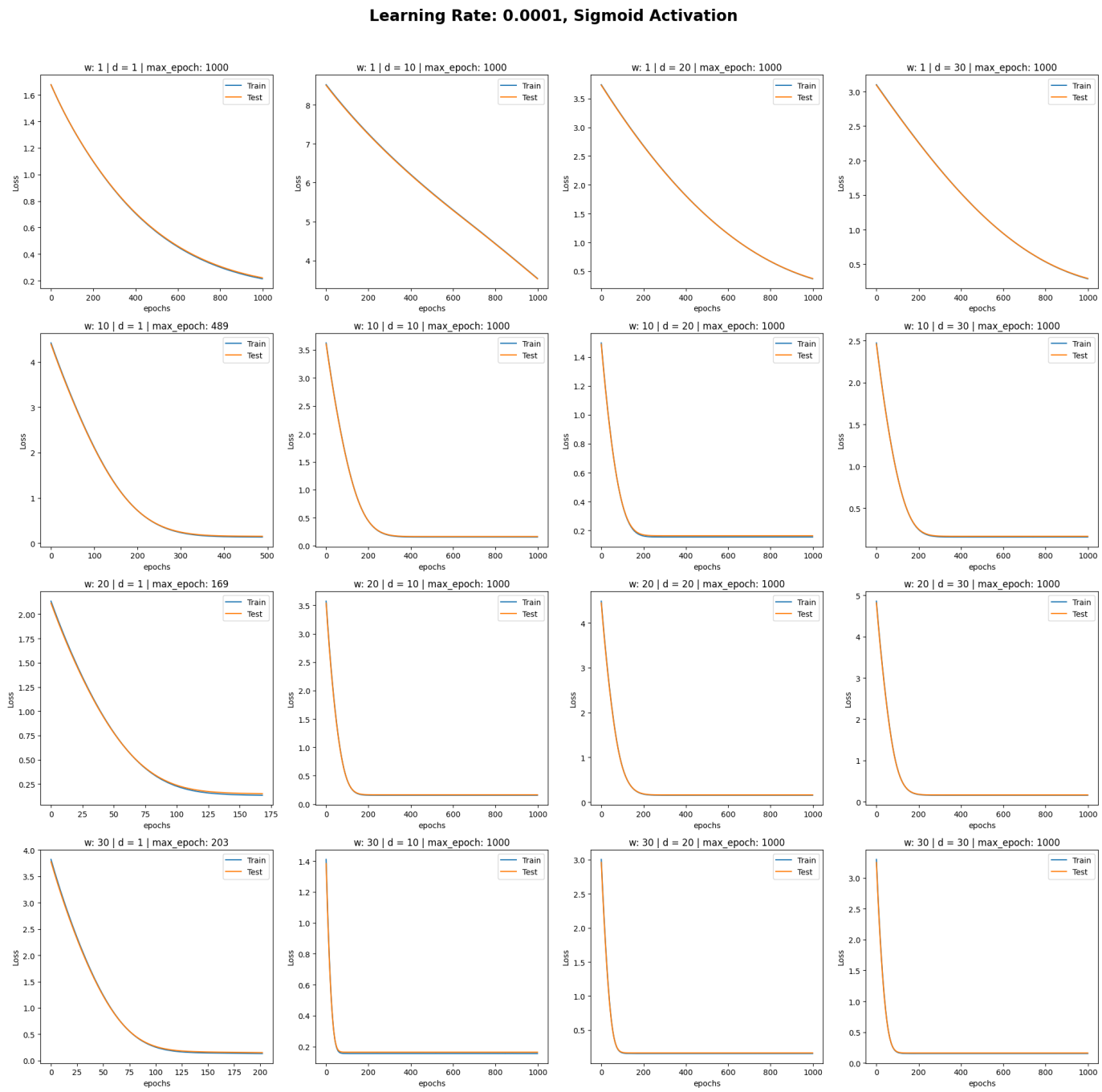

Figure 12: Grid Search for lr=0.0001, Sigmoid Activation

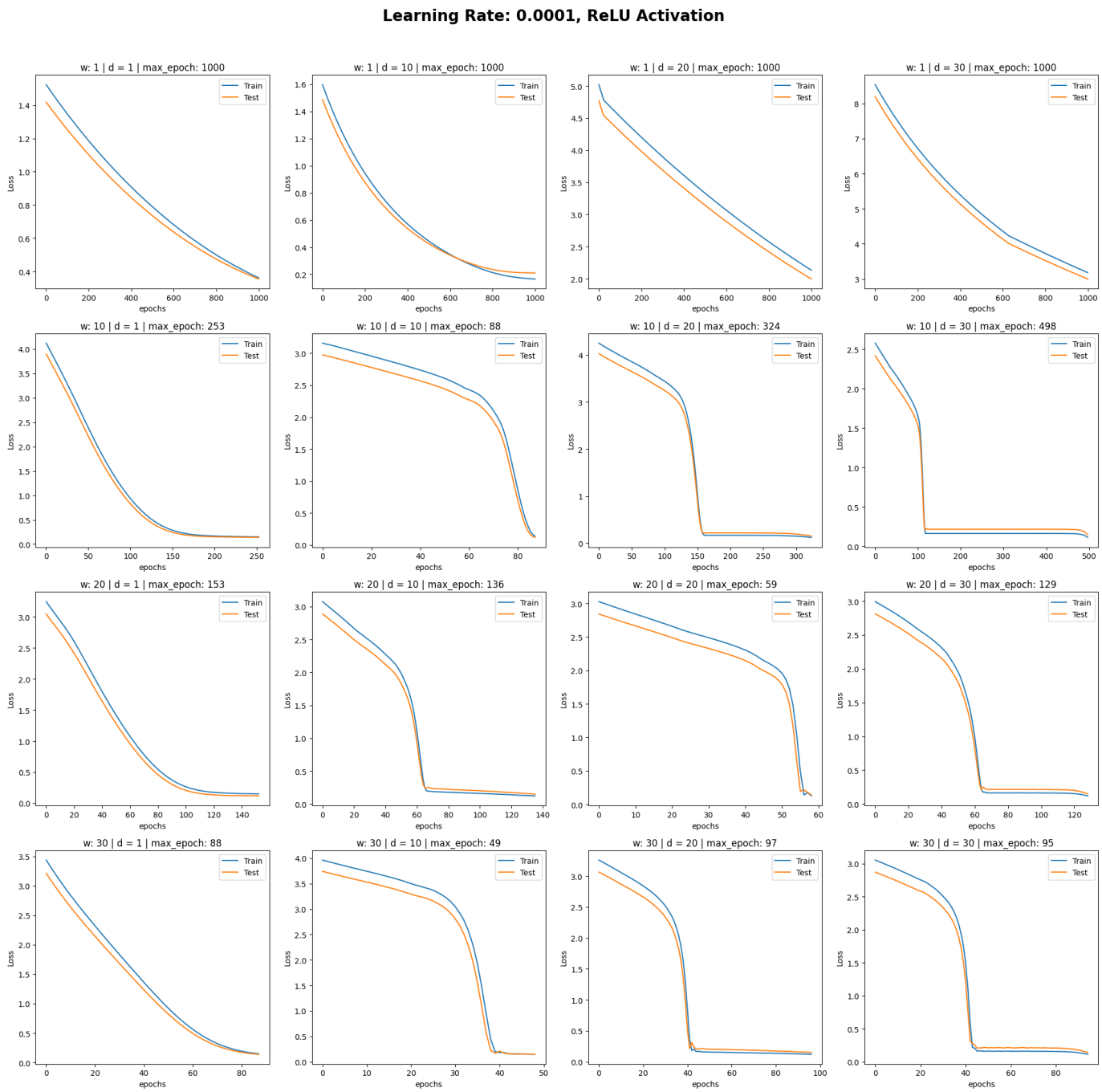

Figure 13: Grid Search for lr=0.0001, Sigmoid ReLU Activation

Justification for Learning Rate $\eta = 0.0001$

With $\eta = 0.0001$, the loss decreases smoothly and monotonically across all widths and depths, indicating highly stable optimization. No oscillations or divergence are observed, even for deeper networks.

However, convergence requires significantly more epochs compared to larger learning rates.

Therefore, while $\eta = 0.0001$ ensures strong numerical stability, it results in slower training.
But ReLU once again proves to be better when it comes to deeper networks.

- $\eta = 0.01$
  - Fast convergence
  - ReLU out performs Simoid
  - Might be Unstable for deeper and wider networks
- $\eta = 0.0001$
  - Very stable optimization
  - Extremely slow convergence
  - Requires a large number of epochs
- $\eta = 0.001$ **(BEST for this case)**
  - Stable across almost all depths and widths
  - Sigmoid out performs ReLU
  - No exploding losses observed
  - Converges at a reasonable speed
  - Smooth training and testing curves

Thus, $\eta = 0.001$ provides the best trade-off between convergence speed and numerical stability.
Also for the $\eta = 0.001$, the lowest epoch convergence happens around **w = 30** and **d = 10**

Comparison of Batch Size, for $\eta = 0.001$

Table 1: Effect of Batch Size on Convergence (Number of Epochs)

	Batch Size					
Dataset	10	30	50	70	90	100
Training (epochs)	10	30	50	39	95	100
Testing (epochs)	10	30	50	39	95	100

Effect of Batch Size on Convergence

From the results, smaller batch sizes converge in fewer epochs, while larger batch sizes require significantly more epochs to meet the convergence criterion.
Smaller batches introduce higher gradient variance, which can help the optimizer escape shallow regions and move faster toward minima. In contrast, larger batches produce smoother but smaller parameter updates, leading to slower convergence in terms of epochs.
Thus, within this experiment, smaller batch sizes lead to faster convergence.

Problem 3

This implementation was done using Pytorch, as **mini batch** approach was used as its the most common when it comes to machine learning. with **BATCH SIZE** = 10.

$$\mathbf{a}_{\ell+1} = \mathbf{a}_\ell + \mathcal{F}(\mathbf{a}_\ell, W_\ell)$$

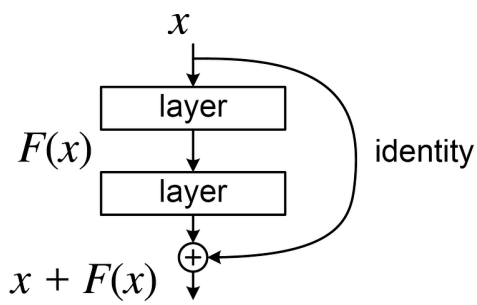

Figure 14: Skip connection representation (skip = 1)

Convergence criteria

for training:

$$|\text{epoch_loss_train} - \text{old_train_loss}| < 0.4 \quad \text{and} \quad \text{epoch_loss_train} < 0.15$$

for testing:

$$|\text{epoch_loss_test} - \text{old_test_loss}| < 0.4 \quad \text{and} \quad \text{epoch_loss_test} < 0.15$$

Main goal

The main goal of this experiment is to make the network intensionally bigger and using residual network try to recover it.

Problem 3 continued on next page. . .

Grid Search

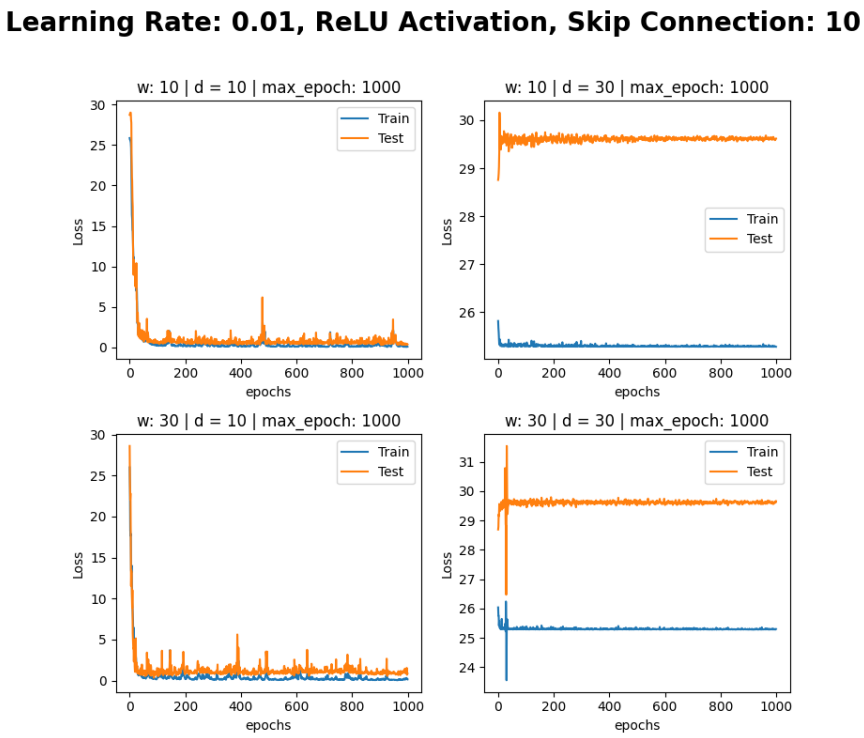

Figure 15: Grid Search for Skip connection of 10

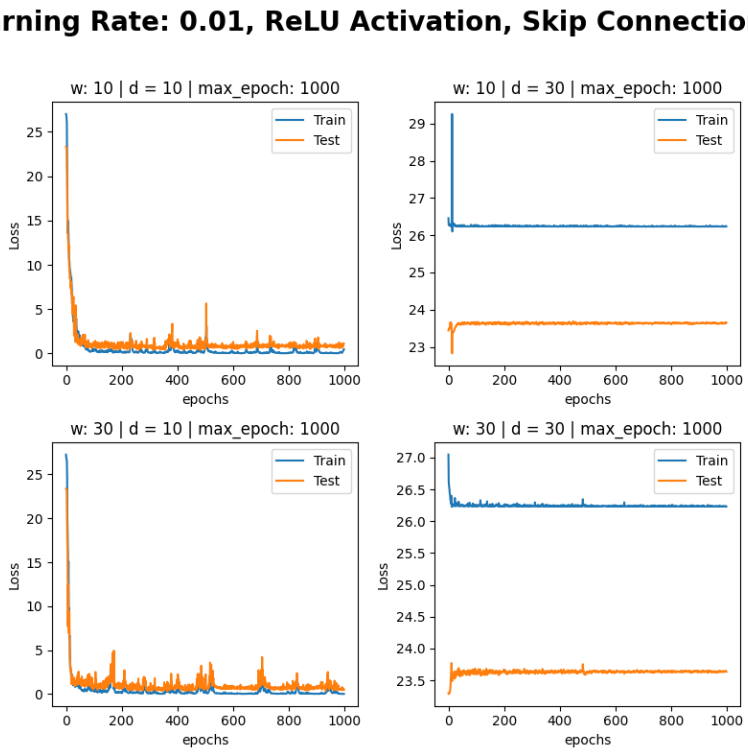

Figure 16: Grid Search for Skip connection of 5

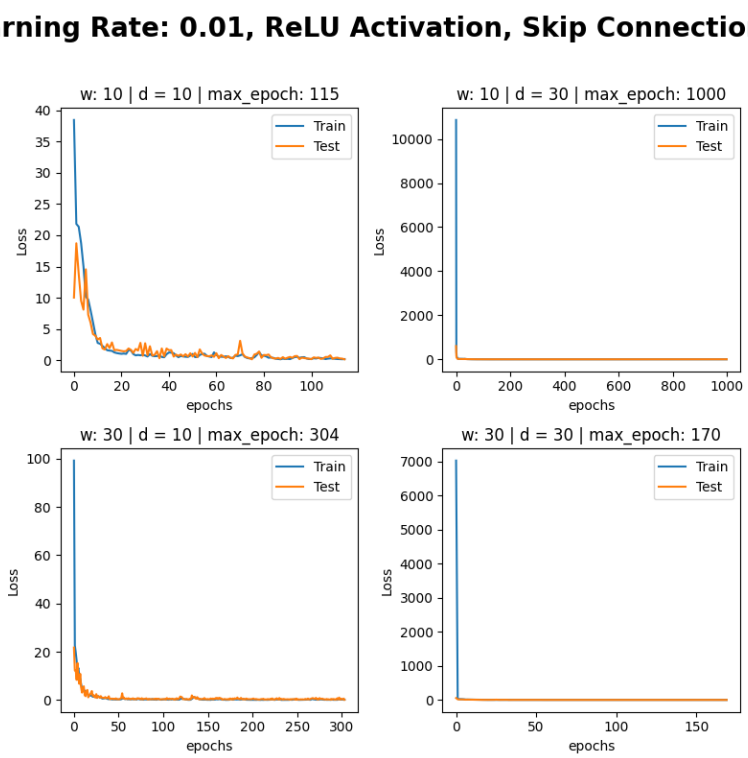

Figure 17: Grid Search for Skip connection of 2

Effect of Skip Connection Frequency on Convergence

For networks with widths 30 and 50, and varying depths, different skip connection intervals were evaluated: skip every 10, 5, and 1 layers. The results show that reducing the skip interval (i.e., increasing the frequency of residual connections) leads to faster convergence, with the ordering:

$$\text{skip } 10 \;\rightarrow\; \text{skip } 5 \;\rightarrow\; \text{skip } 2,$$

where skip 1 converges the fastest.
This behaviour can be explained by improved gradient flow in residual architectures. A residual block is given by

$$\mathbf{a}_{\ell+1} = \mathbf{a}_\ell + \mathcal{F}_\ell(\mathbf{a}_\ell),$$

so during backpropagation,

$$\frac{\partial \mathbf{a}_{\ell+1}}{\partial \mathbf{a}_\ell} = I + \frac{\partial \mathcal{F}_\ell}{\partial \mathbf{a}_\ell}.$$

The presence of the identity term ensures that gradients are preserved across layers.
When skip connections are added more frequently (e.g., skip 1), the effective path for gradient propagation becomes shorter and closer to an identity mapping. This reduces vanishing gradient effects and stabilizes optimization, leading to faster convergence. In contrast, less frequent skip connections (e.g., skip 10) allow longer nonlinear chains between identity mappings, making optimization slower.
Therefore, increasing the frequency of residual connections improves training efficiency in deeper networks.

Disclaimer

No AI aid was used in majority of this assignment except for latex documentation.

Source Code

The complete implementation for this project is available at:

`https://github.com/Mafaz03/Deep-Learning-in-Mechanics`